

DESIGN AND ANALYSIS OF ALGORITHM (CS-327)

LECTURE – 10

BUBBLE SORT ALGORITHM

Bubble sort algorithm is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements. This algorithm is not suitable for large data sets as its worst-case time complexity is quite high. Time complexity of algorithm is $O(n^2)$.

Example :- (For N=5)

Location :- 0 1 2 3 4

Value :-

Pass 1:-
i = 0

50	40	30	20	10
40	50	30	20	10
40	30	50	20	10
40	30	20	50	10
40	30	20	10	50

Pass 2:-
i = 1

40	30	20	10	50
30	40	20	10	50
30	20	40	10	50
30	20	10	40	50
30	20	10	40	50

Pass 3:-
i = 2

30	20	10	40	50
20	30	10	40	50
20	10	30	40	50
20	10	30	40	50
20	10	30	40	50

Pass 4:-
i = 3

20	10	30	40	50
10	20	30	40	50
10	20	30	40	50
10	20	30	40	50
10	20	30	40	50

```
int bubblesort(int arr[ ])
{
    int N=arr.length;
    int temp;
    for(i = 0; i < n-1; i++)
    {
        for(j = 0; j < n-1-i; j++)
        // for optimized algorithm j < n-1-i
        {
            if (arr[j] > arr[j+1])
            {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    return arr; }
```

Another Example :- (For N=5)

Pass 1
i=0

50	40	10	20	30
40	50	10	20	30
40	10	50	20	30
40	10	20	50	30
40	10	20	30	50

Pass 2
i=1

40	10	20	30	50
10	40	20	30	50
10	20	40	30	50
10	20	30	40	50
10	20	30	40	50

Pass3
i=2

10	20	30	40	50
10	20	30	40	50
10	20	30	40	50
10	20	30	40	50

```
int Bubble sor(int arr[])
{
    int N=arr.length;
    int temp ;
    for (i=0 ; i < n-1 ; i++)
        flag=0;
    {
        for (j=0 ; j< n-1; j++)//for best case j < n-1-i
        {
            if(arr[j] > arr[j+1])
            {
                temp = arr[j];
                arr[j]=arr[j+1];
                arr[j+1] = temp;
                flag=1;
            }
        }
        if (flag == 0)
            break ;
    }
    return arr;
}
```

Bubble sort Algorithm	Time Complexity
int Bubble sor(int arr[])	
{	
int n=arr.length;	1
int temp ;	1
for (i=0 ; i < n-1 ; i++)	$1+N-1+1+N-1 = 2N$
{	
for (j=0 ; j < n-1 ; j++)	$1 + x + 1 + x = 2x+2$
	//suppose $j < N-1=x ; x++ =x$
{	
if(arr[j] > arr[j+1])	3x
{	
temp = arr[j];	2x
arr[j]=arr[j+1];	2x
arr[j+1] = temp;	2x
}	
}	
}	
return arr;	1
}	
	$T(N) = 1+1+2N+2x+2+3x+2x+2x+2x+2$
	$T(N) = 11x + 2N + 6$
	$\therefore X = 1 + 2+3 + \dots (N-1)$ in Arithmetic progression
	Considering AP $1+2+3+\dots k = \frac{K(K+1)}{2}$
	$X = \frac{(n-1)(n-1+1)}{2}$
	$T(N) = \frac{11}{2} N^2 - \frac{11}{2}N + 2N + 6$
	Considering big O notation and ignoring constant we consider the term of higher growth.
	$T(N) = O(N^2)$